

Term Project Phase 2

Necla MUTLU CURAL

April 14, 2025

1 Introduction

My covert channel choice was "DNS Subdomain Encoding: Using specific subdomain strings to represent data, which could be retrieved by a specific server".

2 Implementation

2.1 DNS query sender implementation

In the sec container, the DNS query sender crafts DNS query packages given the interval, count, and base domain parameters. The interval parameter determines the time between sending two packets. The count is the number of packages to send, and the base domain is the base domain for DNS queries.

```
def send_dns_query(base_domain, id):
    dns_query = Ether()/IP(dst='insec')/UDP(dport=53)/DNS(id=id, rd=1, qd=DNSQR(qname=base_domain))
    sendp(dns_query)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--interval", type=int, default=100, help="Interval between packets (ms)")
    parser.add_argument("--count", type=int, default=10, help="Number of packets to send")
    parser.add_argument("--base-domain", type=str, default="covert.local", help="Base domain for DNS queries")

    args = parser.parse_args()

    print(f"Sending {args.count} DNS packets to domain *.{args.base_domain} every {args.interval}ms")

    i = 0
    while True:
        send_dns_query(args.base_domain, i)
        time.sleep(args.interval / 1000.0)
        i += 1
```

Figure 1: Sec Container DNS Query Sender Implementation

2.2 DNS query receiver implementation

In the insec container, the DNS query receiver listens to DNS queries and sends mock responses back.

```
qname = dns.qd.qname.decode()
transaction_id = dns.id
# respond back with same dns.id to calculate rtt time in processor
dns_response = DNS(
    id=transaction_id,
    qr=1,
    aa=1,
    qd=dns.qd,
    an=DNSRR(rrname=qname, rdata="1.2.3.4", ttl=60)
)

sent = sock.sendto(bytes(dns_response), address)
```

Figure 2: Insec Container DNS Query Receiver Implementation

2.3 Processor implementation

Processor acts in the middle and captures the DNS queries sent from the sec container and inserts some secret messages to DNS subdomains. It takes 3 parameters: the secret message, chunk size and encoding scheme. The chunk size is the size of the secret message fragment sent in one DNS query. According to encoding scheme choice, those secret message fragments are also encoded.

3 Experiments & results

In the experiments chunk sizes of 4, 8 and 16 are used with 3 different types of encodings rot13, base64 and hex. The effect of the encoding scheme on the capacity and RTT for DNS queries is examined. The message is divided into equal chunks, the size of which is determined by the chunk_size parameter. Then chunks are encoded and sent to the insec container. The RTT time of packets between mitm and insec is measured. Capacity is calculated by dividing the total bytes transferred by RTT. The results can be seen in Figure 4 and Figure 5 below.

ROT13 consistently yields the highest capacity, followed by base64, then hex. This might be due to the fact that ROT13 is a simple letter substitution and keeps character size the same. Base64 expands the data (3 bytes become 4 characters), which slightly reduces efficiency. Hex expands the data the most (1 byte becomes 2 characters), so it performs the worst in capacity.

At very small chunk sizes, more DNS queries are needed to send the full message, which increased overhead. At very large chunk sizes, each individual DNS packet is heavier and might trigger longer processing times or fragmentation, slightly raising RTT. At a medium chunk size (8 bytes), it hits a spot where fewer requests without overloading each one.

Table 1: Performance benchmark results

Encoding Scheme	Chunk Size	Average RTT	CI Margin	Average Capacity (bits)
base64	4	0.0089	0.00023	143.86
hex	4	0.0085	0.00023	123.86
rot13	4	0.0075	0.00030	209.83
base64	8	0.0065	0.00030	393.85
hex	8	0.0061	0.00026	336.13
rot13	8	0.0080	0.00039	370.11
base64	16	0.0078	0.00047	628.61
hex	16	0.0083	0.00041	458.55
rot13	16	0.0078	0.00044	690.38

```

async def message_handler(msg):
    with open(f"{chunk_size}_{encoding_scheme}_rtts.txt", "a") as f:
        nonlocal chunk_index
        subject = msg.subject
        data = msg.data

        packet = Ether(data)

        now = time.time()

        if packet.haslayer(DNS):
            dns_id = packet[DNS].id

            if packet[DNS].qr == 0 and chunk_index < len(chunks):
                # Outgoing query
                secret = chunks[chunk_index]
                qname = packet[DNSQR].qname.decode()

                new_qname = f"{secret}.{qname}"
                packet[DNSQR].qname = new_qname.encode()

                # Reset checksums and lengths
                del packet[IP].len
                del packet[IP].chksum
                if packet.haslayer(UDP):
                    del packet[UDP].len
                    del packet[UDP].chksum

                send_times[dns_id] = now
                print(f"[Chunk {chunk_index}] Payload Size: {len(secret)} bytes")
                chunk_index += 1

            elif packet[DNS].qr == 1:
                if dns_id in send_times:
                    sent_time = send_times.pop(dns_id)
                    rtt = now - sent_time
                    rtts.append(rtt)
                    print(rtt)
                    print(rtt, file=f)
                    print(f"[Response] RTT for DNS ID {dns_id}: {rtt:.6f}
seconds")

                if subject == "inpktsec":
                    await nc.publish("outpktinsec", bytes(packet))
                else:
                    await nc.publish("outpktsec", msg.data)

```

Figure 3: Processor implementation

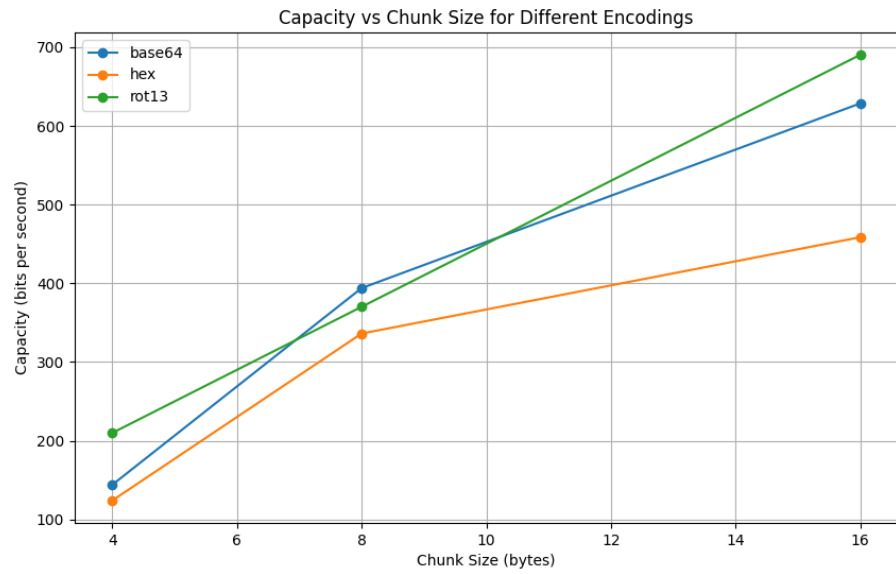


Figure 4: Capacity vs Chunk Size for Different Encodings

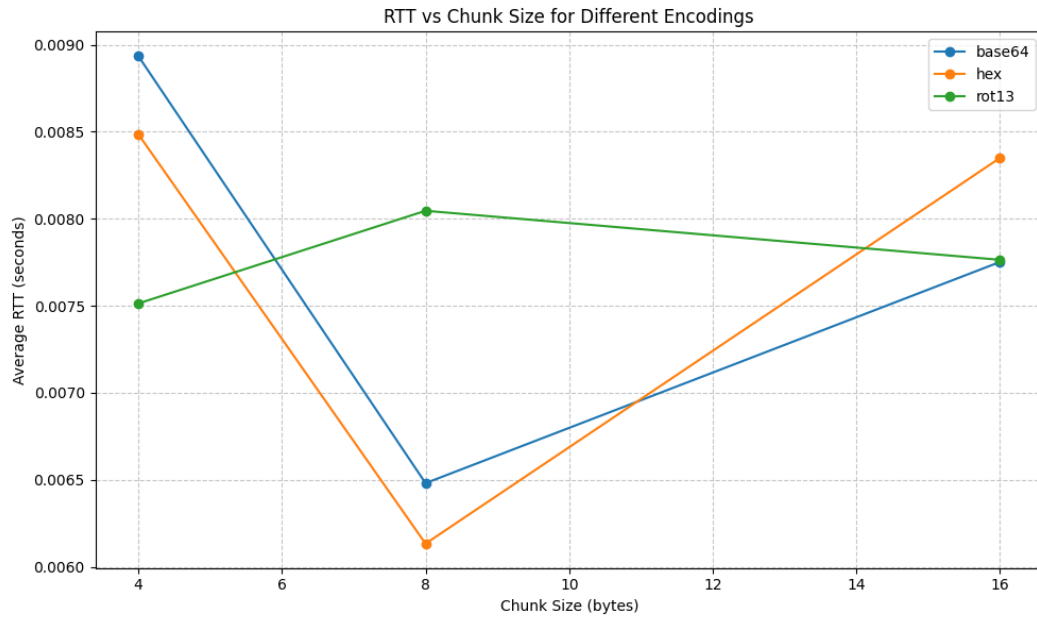


Figure 5: RTT vs Chunk Size for Different Encodings